Entrenar un LLM pequeño para resolver Sudoku: comparación de SFT, DPO y RLVR sobre un entorno verificable

Propuesta inicial – Entrega 1

Lilly Guevara Cristian Hinostroza Juan Moya Dan Villablanca

IIC-3694 Inteligencia Artificial Generativa – 11 de noviembre de 2026

Resumen

Proponemos comparar, sobre un mismo entorno de Sudoku con verificador, cuatro estrategias de post-entrenamiento de Qwen3.5-2B (solo SFT, SFT seguido de RLVR con GRPO, solo RLVR y DPO) junto con el modelo base sin entrenamiento como línea de control, con un presupuesto de cómputo comparable. Como contribución propia, proponemos un currículo de dificultad basado en el número de celdas vacías para el entrenamiento con RLVR, y lo comparamos contra entrenar con dificultades mezcladas. El entorno, los datos y la línea base preliminar provienen del Desafío. Buscamos responder si el SFT previo acelera o limita el aprendizaje con RLVR, cuánto generaliza cada estrategia a la partición de prueba y a niveles de dificultad no vistos, si un currículo por celdas vacías mejora la eficiencia y el resultado final, si lo aprendido se transfiere a tareas de otros entornos del curso, y si aparece *reward hacking*. Documentaremos el ciclo de iteración (entrenar, inspeccionar trazas, corregir el verificador y reentrenar) e incluiremos una auditoría manual de trazas exitosas.

1. Introducción y motivación

El aprendizaje reforzado con recompensas verificables (RLVR) ha pasado a ser una pieza central del post-entrenamiento de modelos de razonamiento [5, 8]. Su ingrediente clave es un verificador automático que sirve como recompensa. Esto lo hace atractivo, pero también frágil: un verificador imperfecto puede ser explotado por la política [16]. Además, hay evidencia mixta sobre si RLVR agrega capacidades o solo afina las que el modelo ya posee [21], y sobre el rol del SFT previo: Chu et al. [4] observan que el SFT tiende a memorizar mientras que RL generaliza mejor a variantes no vistas, aunque el SFT estabiliza el formato de salida y facilita el RL posterior, como en el arranque en frío de DeepSeek-R1 [5].

Estas preguntas son difíciles de estudiar a escala de laboratorio de frontera, pero se pueden abordar con un modelo pequeño y un entorno barato de verificar. En el Desafío construimos un harness de Sudoku (4×4, 6×6 y 9×9, tres niveles de dificultad) cuya respuesta se verifica por construcción. El Sudoku es un problema de razonamiento con restricciones, de varios pasos, que Qwen3.5-2B resuelve mal: en nuestra línea base obtiene un `pass@1` de 6,9 % y ningún acierto en 9×9. Este proyecto usa ese entorno para comparar estrategias de entrenamiento de forma controlada. Entrenaremos solo con 4×4 y 6×6; los 9×9 quedan fuera del entrenamiento y se usarían únicamente para evaluar.

Objetivo y contribuciones. Buscamos comparar de forma controlada, sobre este entorno, los cinco caminos de entrenamiento que define el enunciado [11]. Las contribuciones esperadas son: (i) una comparación de SFT, DPO y RLVR a cómputo comparable, con `pass@1` y `pass@`k por nivel de dificultad; (ii) un currículo por número de celdas vacías para GRPO, evaluado contra la mezcla uniforme de dificultades, y (iii) un registro del ciclo de iteración y una auditoría manual de trazas exitosas para detectar *reward hacking*.

2. Revisión bibliográfica

Entornos verificables. Reasoning Gym [17] ofrece generadores de tareas con verificadores y dificultad configurable; nuestro generador de Sudoku adapta el suyo. AgentGym [19] sigue la misma filosofía para agentes con herramientas.

Algoritmos. PPO [13] es la base del RLHF de InstructGPT [10]. GRPO [15] elimina el crítico y estima la ventaja comparando las recompensas de un grupo de respuestas a un mismo prompt; así se usó en DeepSeek-R1 [5]. Trabajos posteriores analizan sesgos de GRPO, como la normalización por longitud y por desviación estándar [9], y proponen mejoras prácticas, como filtrar grupos sin señal y recortar de forma asimétrica [20]. DPO [12] optimiza directamente sobre pares de preferencia, sin modelo de recompensa ni muestreo en línea, y es de bajo costo. Para SFT y DPO con pocos recursos usaremos LoRA [6] sobre las implementaciones de TRL [18].

Currículo. El aprendizaje por currículo presenta ejemplos de menor a mayor dificultad [2]. En RLVR para LLM, Kimi k1.5 usa muestreo por currículo y priorizado [7], y DAPO filtra grupos donde todas las respuestas reciben la misma recompensa [20]. Nuestro entorno permite controlar la dificultad con una variable explícita y barata, el número de celdas vacías, lo que facilita estudiar el currículo de forma controlada.

SFT frente a RL. Chu et al. [4] comparan ambos post-entrenamientos en un juego de cartas aritmético y en una tarea de navegación: RL generaliza a variantes no vistas de las reglas, mientras que SFT tiende a memorizar; aun así, el SFT previo estabiliza el formato de salida y permite que RL obtenga sus ganancias. Nuestros caminos 2 a 4 ponen a prueba esta observación en otro dominio y con un modelo de lenguaje pequeño.

Sudoku y LLM. Sudoku-Bench [14] reúne 100 variantes de Sudoku y reporta que los LLM de frontera resuelven menos del 15 % sin ayuda. A diferencia de ese trabajo, que solo evalúa, usamos Sudokus clásicos de tamaño y dificultad controlados como entorno de entrenamiento.

Qué aporta RLVR. Yue et al. [21] reportan que, medido con `pass@`k para k grande [3], los modelos entrenados con RLVR no superan al modelo base. Esto motiva reportar `pass@`k además de `pass@1`, y estudiar en qué zona de dificultad hay señal.

Reward hacking y auditoría. Skalse et al. [16] formalizan el *reward hacking*. En código, SWE-Bench+ [1] mostró que una parte importante de los aciertos aparentes no era genuina, lo que motiva la auditoría manual de trazas exitosas que pide el enunciado [11].

Nuestra posición. A diferencia de estos trabajos, combinamos en un mismo entorno barato de verificar la comparación de los cinco caminos de entrenamiento, un currículo con una variable de dificultad explícita y una auditoría de los aciertos.

3. Entorno a utilizar

Emplearemos el *harness* desarrollado por nuestro grupo para el Desafío (cuyo repositorio será publicado próximamente) para entrenar el modelo `Qwen/Qwen3.5-2B`.

- **Interfaz:** `reward(prompt, completion) ->float`. El modelo puede razonar libremente antes de responder, pero su salida debe finalizar con un único bloque `<answer>...</answer>` que contenga el tablero como una matriz JSON de $N \times N$ enteros. El *parser* invalida la respuesta (format $= 0$) si existe texto posterior a dicho bloque, si aparece más de una etiqueta `<answer>` o si esta se encuentra anidada dentro del bloque de razonamiento (`<think>`). De este modo, se evita que el modelo burle al verificador entregando múltiples soluciones candidatas.
- **Tareas:** *Splits* de *train/eval/test* disjuntas entre sí, compuestas por tableros con solución única distribuidos en tres tamaños y tres niveles de dificultad por tamaño.
- **Función de recompensa:** Se define como

$$\text{reward} = \text{format} \cdot (w_{\text{fill}} \cdot \text{fill} \cdot \text{clues_frac} + w_{\text{solve}} \cdot \text{solved}),$$

  sujeta a $w_{\text{fill}} + w_{\text{solve}} = 1$ (por defecto, 0.5 cada uno). Dado que todos los factores pertenecen al intervalo $[0, 1]$, la recompensa resultante también está acotada en dicho rango. Cada componente evalúa un aspecto específico:

  - format $\in \{0, 1\}$: Toma el valor 1 si la salida respeta estrictamente el contrato estructural descrito; en caso contrario, anula la recompensa total.
  - clues_frac $\in [0, 1]$: Proporción de pistas iniciales del tablero que se mantienen intactas, penalizando errores al copiar la entrada.
  - fill $\in [0, 1]$: Proporción de celdas originalmente vacías que coinciden con la solución única. Actúa como crédito parcial por avance, contabilizando como error tanto los ceros como los valores fuera de rango.
  - solved $\in \{0, 1\}$: Toma el valor 1 si el tablero cumple las reglas del Sudoku (filas, columnas y subcuadrículas válidas) y conserva todas las pistas originales, aceptando cualquier solución válida.

  Así, una solución completa otorga reward $= 1$, mientras que un tablero bien formateado que conserva las pistas pero acierta solo 40 de 50 celdas vacías obtiene $0.5 \cdot 0.8 = 0.4$. Como línea de comparación (*ablation*), evaluaremos también una variante binaria (reward $=$ format $\cdot$ solved) para aislar el impacto del crédito parcial.
- **Datos generados:** Trazas de razonamiento correctas para SFT y pares de respuestas correcta/incorrecta para DPO, generados sobre el *split* de entrenamiento utilizando `Qwen3.5-9B` como modelo profesor y filtrados mediante el verificador.

***Baseline* preliminar.** Para cada tarea se muestrean n respuestas independientes del modelo (con temperatura $T > 0$) y se evalúan con el verificador; a partir de la proporción de aciertos se calcula el estimador insesgado de `pass@`k (con $k \leq n$) siguiendo a Chen et al. [3]. En una evaluación inicial del modelo de 9B para el Desafío ($n = 8$ muestras en 10 tareas de validación por combinación de tamaño y dificultad, totalizando 80 respuestas por celda), solo 12 de las 90 tareas presentaron resultados mixtos (combinación de aciertos y fallos). Estas se concentraron exclusivamente en los tableros de 4×4 (en todos sus niveles) y en los de 6×6 de dificultad *easy* y *medium*. Por el contrario, el modelo no resolvió ninguna instancia en 6×6 *hard* ni en los tableros de 9×9. Esta ausencia de señal de aprendizaje, sumada al alto costo computacional de generar datos y entrenar sobre tableros de 9×9, justifica descartar dicho tamaño en la etapa de entrenamiento. Actualmente estamos recalculando estas métricas con un conjunto de datos ampliado, por lo que los valores reportados son provisionales

y serán actualizados en la Entrega 2.

4. Propuesta de investigación

4.1. Preguntas e hipótesis

P1. ¿El SFT previo acelera o limita el aprendizaje con RLVR? *H1:* SFT→RLVR converge más rápido que solo RLVR en la zona aprendible, pero con `pass@`k final similar o menor para k grande.
P2. ¿Cuánto generaliza cada estrategia a la partición de prueba, a niveles no vistos y a un tamaño de tablero no visto (9×9)? *H2:* RLVR generaliza mejor que SFT y DPO a niveles no vistos; SFT tiende a memorizar el formato de las trazas del profesor.
P3. ¿Aparece *reward hacking* y bajo qué condiciones? *H3:* aparecerá más con recompensa parcial que con la binaria, por ejemplo respuestas que copian pistas y adivinan el resto, o que explotan el formato.
P4. ¿Un currículo por número de celdas vacías mejora el entrenamiento con RLVR frente a mezclar dificultades? *H4:* un currículo adaptativo alcanza la misma recompensa con menos pasos y llega a niveles que el entrenamiento con mezcla uniforme no resuelve (por ejemplo 6×6 *hard*), porque mantiene los grupos de GRPO en la zona con señal.
P5. ¿Lo aprendido en Sudoku se transfiere, o se pierde, en tareas de razonamiento de otros entornos? *H5:* el SFT degrada más el desempeño fuera de dominio que RLVR, que se mantiene más cerca del modelo base.

4.2. Experimento central

Los cinco caminos (el modelo base como control y cuatro estrategias de entrenamiento) se evalúan sobre el mismo entorno, con presupuesto de cómputo comparable (criterio en la sección 5).

#	Camino	Descripción
1	Base (control)	Qwen3.5-2B sin entrenamiento adicional.
2	SFT	SFT con LoRA sobre trazas correctas del profesor.
3	SFT→RLVR	Camino 2 seguido de GRPO con el verificador como recompensa.
4	RLVR	GRPO directamente desde el modelo base.
5	DPO	DPO con LoRA sobre pares correcta/incorrecta.

4.3. Diseño y ablations

- **Datos de entrenamiento y evaluación.** Se entrena solo con 4×4 y 6×6 (todos los niveles, ya que el currículo busca llegar a 6×6 *hard*); los caminos 2 a 5 usan una mezcla fija de dificultades. Los 9×9 se excluyen del entrenamiento. Por la línea base, la zona con señal es 4×4 y 6×6 *easy*/*medium*. *[Por decidir: confirmar la mezcla tras la nueva línea base]*
- **Evaluación en 9×9.** Si se mantiene, mide generalización a un tamaño no visto. Con la línea base actual se espera `pass@1` cercano a cero, por lo que podría aportar poca información. *[Por decidir: mantener 9×9 en la evaluación o descartarlo del todo]*
- **Ablations (RLVR).** Tamaño de grupo G, coeficiente KL, forma de la recompensa (binaria vs. parcial) y mezcla de dificultades. *[Por decidir: cuáles son viables con el cómputo]*

- **Contribución (nivel 3): currículo por celdas vacías.** En lugar de los tres niveles gruesos, se usa el número de celdas vacías como eje continuo de dificultad. Se entrena GRPO con tres variantes de muestreo de tareas: (a) *mezcla uniforme* sobre todo el rango (baseline); (b) *currículo fijo*, con un calendario creciente de celdas vacías; (c) *currículo adaptativo*, que avanza al siguiente rango cuando el `pass@1` de entrenamiento reciente supera un umbral τ y retrocede si cae bajo uno inferior. Así se llega más allá de la zona actual, hasta 6×6 *hard*. La comparación es contra GRPO base, con igual cómputo, y se reportan curvas de recompensa y `pass@`k por número de celdas vacías. *[Por decidir: umbral τ, calendario, tamaño de ventana]*
- **Eje de dificultad entre tamaños.** Las celdas vacías no son comparables entre tableros (12 vacías es el máximo en 4×4, pero en 6×6 corresponde a *easy*). Opciones: currículo por tamaño y dentro de cada tamaño por celdas vacías, o un orden global por dificultad empírica (`pass@1` de la línea base). *[Por decidir: elegir el eje]*
- **Soporte en el entorno.** El generador ya produce tareas por rangos de celdas vacías (`LEVELS`) y la partición de prueba admite conteos fijos (`TEST_CELLS`). El conjunto actual (`data/sudoku`) tiene solo 50 tareas de entrenamiento por tamaño y nivel (150 por tamaño). Para un currículo fino (4×4 tiene 9 conteos de celdas y 6×6 tiene 17) hay que generar más tareas, con un nivel por conteo, y escribir un muestreador de tareas para el currículo. La partición de prueba usa conteos fijos y no cambia.
- **Métricas.** `pass@1` y `pass@`k por nivel de dificultad y por número de celdas vacías, con el estimador insesgado de Chen et al. [3], en la partición de prueba, y varias semillas (≥ 3 si el cómputo lo permite). Todos los resultados se reportan con la *versión final* del verificador.
- **Evaluación en otros entornos.** Los modelos de los cinco caminos se evaluarán, sin entrenar en ellos, con harness de otros grupos del curso, para medir transferencia u olvido respecto al modelo base. Como esos entornos se publicarán durante esta semana, los elegiremos una vez disponibles, con este criterio: (i) que respeten la interfaz común `reward(prompt, completion) ->float`; (ii) que sean de razonamiento o de uso de herramientas y no de memorización; (iii) que el modelo base tenga `pass@1` distinto de cero, para que la comparación sea informativa; y (iv) que la inferencia sea de bajo costo. Se reporta `pass@1` y `pass@`k como en Sudoku, y se compara cada camino contra el modelo base. Si no hay harness de otros grupos utilizables, usaremos como respaldo tareas públicas de Reasoning Gym [17] distintas del Sudoku. *[Por decidir: cuál o cuáles harness; cuántas tareas por entorno]*
- **Auditoría manual.** Muestra de trazas exitosas de cada camino, revisada a mano para estimar qué fracción de los aciertos es genuina (razonamiento válido), al estilo de Aleithan et al. [1]. *[Por decidir: tamaño de la muestra y quién audita]*

4.4. Ciclo de iteración y bloques del sistema

Registraremos cada iteración: qué se observó en las trazas, qué se corrigió en el entorno o el verificador y su efecto. La figura 1 resume el sistema.

5. Cómputo y riesgos

Los experimentos se ejecutarán en un clúster equipado con una GPU NVIDIA RTX A6000 (48 GB). Mientras que SFT y DPO con LoRA presentan un bajo costo computacional, GRPO es significativamente más exigente debido al muestreo en línea, por lo que acotaremos tanto la longitud máxima de las respuestas como la complejidad del entorno.

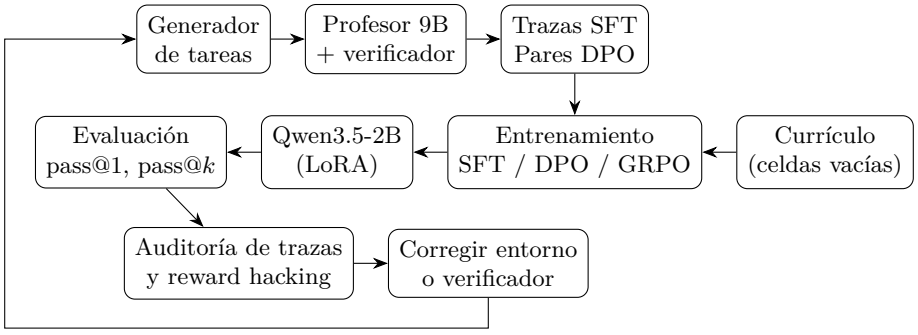


Figura 1: Diagrama de bloques tentativo. El verificador también es la recompensa del RL, y el currículo decide qué tareas ve el entrenamiento.

Comparación a cómputo equivalente. Los métodos evaluados tienen estructuras de costo distintas: SFT y DPO operan sobre conjuntos de datos estáticos, mientras que GRPO genera G respuestas por *prompt* antes de cada actualización. Para garantizar una comparación justa, establecemos los siguientes criterios:

- **Presupuesto común (B):** Se define en términos de tokens procesados por el modelo objetivo (tanto generados como de entrenamiento) y horas de GPU en la A6000, acotado por la capacidad de ejecución de GRPO. El esquema secuencial SFT→RLVR distribuye este mismo presupuesto B entre sus dos etapas.
- **Curvas frente a cómputo acumulado:** Se evalúan *checkpoints* en intervalos fijos de B para graficar `pass@1` en función de los tokens y las horas de GPU. De este modo, la equivalencia computacional se observa directamente en la curva de aprendizaje sin depender de un único criterio de parada.
- **Configuración homogénea:** Todos los enfoques comparten el mismo rango de LoRA, la misma longitud máxima de respuesta y un presupuesto de evaluación idéntico.
- **Contabilidad separada del costo de datos:** La generación de trazas para SFT y de pares para DPO requiere inferencia del modelo profesor (9B), un gasto ausente en RLVR puro. Este costo se reportará por separado y no se descontará de B.

Calibración del presupuesto B. Fijamos $B = 15$M de tokens generados por ejecución de GRPO (equivalente a unos 200 pasos con 16 entradas y $G = 8$), con un límite máximo de 5 horas de GPU en la A6000 por ejecución. Con la longitud de respuesta estimada, este margen permite realizar aproximadamente 16 ejecuciones completas de GRPO. Dado que SFT y DPO consumen solo una fracción de B, reportaremos su gasto real sin aplicar compensaciones artificiales. Por su parte, SFT→RLVR descontará el costo de la etapa supervisada de su presupuesto total B, y el costo de inferencia del profesor (`Qwen3.5-9B`) se mantendrá aislado del análisis comparativo. Estas estimaciones se recalibrarán en la Entrega 2 utilizando el *throughput* medido en la A6000 y la longitud media de respuesta del nuevo *baseline*.

Análisis de riesgos. A continuación, detallamos los principales riesgos técnicos y experimentales junto con sus mecanismos de detección y mitigación:

Ausencia de señal en RL. GRPO actualiza la política comparando las recompensas relativas de las G respuestas generadas para un mismo *prompt*. Si el modelo falla o acierta sistemáticamente en todas las muestras de un grupo, las recompensas son idénticas, la ventaja se anula y el gradiente desaparece. *Detección:* Monitoreo de la proporción de grupos con recompensa constante y estan-

camiento en la curva de recompensa inicial. *Mitigación:* Restringir el entrenamiento a instancias dentro de la zona con señal, filtrar grupos sin varianza y aplicar un precalentamiento mediante SFT antes de RLVR (esquema SFT→RLVR) si el modelo base carece de aciertos iniciales.

Incumplimiento del formato de salida. Dado que el factor format multiplica toda la función de recompensa, un modelo que omita o duplique el bloque `<answer>` recibirá siempre cero, anulando la señal de aprendizaje incluso cuando el razonamiento sea válido. Este problema se agrava si la generación se trunca por alcanzar el límite de tokens antes de cerrar la etiqueta. *Detección:* Seguimiento de las tasas de formato válido y de truncamiento en cada experimento. *Mitigación:* Aumentar la longitud máxima de generación, incluir ejemplos de formato en el *prompt* (*few-shot*) o inicializar el entrenamiento desde un modelo ajustado con SFT.

Reward hacking. La política podría descubrir atajos que incrementen la recompensa sin resolver realmente el Sudoku; por ejemplo, bajo recompensa parcial, copiar las pistas y completar el resto al azar otorga un puntaje positivo, o bien podría explotar vulnerabilidades del *parser*. *Detección:* Inspección periódica de trazas con alta recompensa, análisis de divergencias entre la recompensa media y la tasa real de resolución (solved) y auditorías manuales. *Mitigación:* Diseño de pruebas unitarias adversariales (como múltiples bloques `<answer>`, texto posterior al cierre, tableros vacíos o rellenos aleatorios) que deban puntuar cero o mínimo; ante cualquier falla detectada, se parchará el verificador, se incorporará el caso a la batería de pruebas y se reentrenarán los modelos afectados.

Estancamiento o ineficacia del currículo. Un umbral de avance τ demasiado exigente bloqueará el currículo adaptativo en las tareas más simples, mientras que uno muy laxo desplazará el entrenamiento hacia instancias sin señal. Asimismo, un resultado nulo (donde el currículo no supere al muestreo uniforme) es científicamente válido, pero debe distinguirse de una mala calibración. *Detección:* Registro continuo de la distribución de dificultades muestreadas por paso. *Mitigación:* Calibrar τ apoyándose en los resultados de la línea base y utilizar un calendario fijo (incremento de dificultad predeterminado por etapas) como alternativa determinista.

Escasez de instancias de entrenamiento. El conjunto actual dispone de 50 tareas por combinación de tamaño y nivel (450 en total). Bajo el presupuesto B, el modelo recorrerá estas instancias múltiples veces, lo que conlleva el riesgo de sobreajuste por memorización de tableros específicos en lugar de generalización. *Detección:* Brecha creciente en `pass@1` entre los conjuntos de entrenamiento y prueba. *Mitigación:* Generar instancias adicionales y, de ser necesario, sintetizar nuevas trazas con el modelo profesor.

Indisponibilidad o incompatibilidad de entornos externos. Los *harnesses* desarrollados por otros grupos aún no están publicados y podrían presentar barreras técnicas o un costo de evaluación prohibitivo. *Mitigación:* Preseleccionar al menos dos entornos candidatos, validar su interfaz y el `pass@1` del modelo base antes de integrarlos, utilizar tareas de *Reasoning Gym* como alternativa de respaldo y relegar esta evaluación a un plano secundario si surgen restricciones de tiempo.

Insuficiencia de cómputo. Si el presupuesto de GPU resulta insuficiente para cubrir toda la matriz experimental, se perderán repeticiones con distintas semillas aleatorias, reduciendo el poder estadístico para distinguir mejoras reales del ruido. *Mitigación:* Priorizar múltiples semillas en los enfoques centrales (SFT→RLVR, RLVR puro y las variantes de currículo), recortar estudios de ablación secundarios antes que semillas y guardar *checkpoints* frecuentes para reanudar ejecuciones interrumpidas.

6. Cronograma

Fecha	Hito
11 nov	Entrega 1: esta propuesta.
11–25 nov	Línea base final (cluster), datos SFT/DPO, entrenamiento de SFT y DPO, primeras curvas de recompensa GRPO.
25 nov	Entrega 2: informe de avance (línea base, implementación inicial, curvas de recompensa). Elegir los harness externos.
25 nov – 9 dic	Caminos 3 y 4 completos, currículo y ablations, auditoría manual, redacción del paper y del póster.
9 dic	Entrega 3: paper (7 páginas), póster y video.

Referencias

[1] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for LLMs. *arXiv preprint arXiv:2410.06992*, 2024.

[2] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning (ICML)*, 2009.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. In *International Conference on Machine Learning (ICML)*, 2025.

[5] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[7] Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.

[8] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. TÜLU 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[9] Zichen Liu, Changyu Chen, Wenjun Li, et al. Understanding R1-Zero-Like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[10] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[11] Pontificia Universidad Católica de Chile. Proyecto de Investigación: IIC-3694 Inteligencia Artificial Generativa, 2026. Enunciado del curso.

[12] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[14] Jeffrey Seely, Yuki Imajuku, Tianyu Zhao, Edoardo Cetin, and Llion Jones. Sudoku-Bench: Evaluating creative reasoning with Sudoku variants. *arXiv preprint arXiv:2505.16135*, 2025.

[15] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[16] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[17] Z. Stojanovski, O. Stanley, J. Sharratt, R. Jones, A. Adefioye, J. Kaddour, and A. Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.24760.

[18] Leandro von Werra, Younes Belkada, Lewis Tunstall, et al. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

[19] Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, et al. AgentGym: Evaluating and training large language model-based agents across diverse environments. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[20] Qiying Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[21] Yang Yue, Zhiqi Chen, Rui Lu, et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.